HTTP QUERY method

The problem that existed for 30+ years

Suppose you're building an e-commerce API.

Simple search:

```
GET /products?category=laptop
```

Perfect

But now your frontend allows:

- category
- price range
- multiple brands
- stock status
- ratings
- availability
- sorting
- pagination
- nested filters

Your request becomes something like

```
GET /products?
category=laptop&
brand=dell&
brand=hp&
price[min]=50000&
price[max]=120000&
rating>=4&
inStock=true&
sort=-price&
page=4
```

Eventually URLs become:

- huge
- ugly
- hard to read
- difficult to generate
- sometimes exceed limits imposed by browsers, proxies, servers, or gateways
- difficult to cache consistently when filters become deeply nested

Why not just send JSON in GET?

```
GET /products

{
   "category":"laptop",
   "price":{
      "min":50000,
      "max":120000
   }
}
```

Looks nice.

Unfortunately...

HTTP never defined semantics for a GET request body.

That means:
Different software behaves differently.

Some servers:
- accept it

Others:

- ignore it
- strip it
- reject it
- proxies remove it
- firewalls block it
- caches don't understand it

Because of this, GET with body has never become interoperable. The HTTP working group intentionally did not redefine GET to support bodies because that would break decades of existing infrastructure.

So everyone started using POST...

Instead of

```
GET /products?...lots of filters...
```

people wrote

```
POST /products/search

{
  ...
}
```

Every major API does this.

Examples include:

- Elasticsearch
- GraphQL
- many REST APIs
- internal enterprise APIs

But POST creates another problem.

Why POST is not the right solution

Imagine this request

```
POST /products/search

{
   "category":"Laptop"
}
```

But HTTP semantics say:

The server simply searches.

Nothing changes.

No data is created.

No database update.

So logically this is still a read operation.

POST means

"Process this request."

Processing may:

- create resources
- modify resources
- trigger actions
- send emails
- charge credit cards
- anything

POST is not safe.
POST is generally not idempotent.
HTTP infrastructure therefore assumes:
This request may change server state.

That's the mismatch QUERY fixes.

What exactly is QUERY?

QUERY says

"I'm sending a request body, but this is still only reading data."

It combines

GET

- read operation
- safe
- idempotent

with

POST

- request body

So QUERY is essentially

GET semantics + POST body

```
QUERY /products

Content-Type: application/json

{
   "category":"Laptop",
   "brands":["Dell","HP"],
   "price":{
      "min":50000,
      "max":120000
   },
   "sort":"price"
}
```

DEMO

Why didn't they simply improve POST?

Because HTTP methods carry meaning, not just syntax.

If every read request uses POST:

A proxy cannot know

Is this creating data?
Is this only searching?
Can I retry it?
Can I cache it?
Can I prefetch it?

Everything becomes ambiguous.

QUERY makes the intent explicit.

Safe

QUERY is officially defined as safe.
Meaning:
   It should not intentionally modify server state.
   Like GET.

Example

```
QUERY /search
```

Allowed

- Search
- Filter
- Analytics query
- Report generation

Not allowed

- Create User
- Delete Product
- Update Order

Idempotent

QUERY is also idempotent.

Meaning

QUERY
QUERY
QUERY
QUERY

100 times

=

same effect as once.

Because nothing changes.

This allows automatic retries if a connection fails.

Cacheable

QUERY responses can be cached.

This is huge.

POST responses are generally not treated as cacheable by default.

QUERY explicitly supports caching, with caches incorporating the request body into the cache key

Request Body

QUERY supports any body format.

Examples

JSON

```
{
 "category":"Laptop"
}
```

XML

```
<Search>
...
</Search>
```

Form

```
q=laptop
```

Binary

Protocol Buffers

Almost Anything.

URI stays clean

Instead of

```
GET /search?
category=laptop&
brand=hp&
...
```

You get

```
QUERY /search
```

Body

```
{
...
}
```

Much cleaner.

Does QUERY replace GET?

No.

Use GET whenever parameters are simple.

Example

```
GET /users/10
GET /products?page=2
GET /orders/100
```

GET remains the best choice.

QUERY is for complex read-only queries.

Does QUERY replace POST?

No.

POST still handles:

- login
- uploads
- payments
- resource creation
- commands
- workflows
- webhooks

QUERY only replaces the pattern:

```
POST /search
```

when that POST is actually a read.

New Accept-Query header

Servers can advertise which query body formats they support using the new Accept-Query response header.

Example:

```
Accept-Query: application/json
```

or

```
Accept-Query:
application/json,
application/xml
```

Clients can inspect this to know which content types are accepted for QUERY

Status codes

QUERY uses normal HTTP response codes:

- 200 OK
- 204 No Content
- 304 Not Modified (with conditional requests)
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

Current adoption (mid-2026)

The standard is new. RFC 10008 was published in June 2026, so ecosystem support is still emerging.

Expect gradual adoption in:

- browsers
- API gateways
- proxies
- frameworks
- HTTP client libraries
- cloud providers

Many existing APIs will continue using POST /search for some time because changing HTTP methods is a breaking API chang

Should you use QUERY today?

For new internal APIs, it's worth tracking if your framework and infrastructure support it.

For public production APIs, POST /search remains the most interoperable choice today unless your entire stack supports QUERY.

Over the next few years, QUERY is expected to become the semantically correct choice for complex, read-only operations as tooling catches up